Octobre 2026

Rapport de TP

Detection Engineering & Dashboards (SOC L1)

Pape Aldiouma SARR
L3 Cybersecurite - Module : Ethical Hacking / SOC

Rapport de travaux pratiques : TP 4, Detection Engineering & Dashboards. Parcours SaaS : RBAC, import de regles et dashboards, alerting, tuning, regles maison, EQL et detection-as-code.



Republique du Senegal, Un Peuple, Un But, Une Foi
Ministere de l'Enseignement Superieur, de la Recherche et de l'Innovation
Direction Generale de l'Enseignement Superieur Prive
Institut Superieur de Management
Annee academique 2026-2027

Table des matieres

Partie I : Presentation du travail....3
Partie II : Espace et RBAC....4
Partie III : Import des regles et dashboards....6
Partie IV : Alerting et tuning....8
Partie V : Regles maison (Etape 6)....10
Partie VI : EQL et detection-as-code....15
Partie VII : Defis (autonomie)....17
Conclusion....20

Partie I : Presentation du travail

Contexte

A l'issue du TP2, la plateforme SOC-LAB-GLACIER (Elastic Cloud Hosted) contient des donnees simulees ingerees et interrogeables (1505 documents repartis sur deux index). Ce TP4 transforme ces capacites de recherche en detection operationnelle : importer des regles de detection et des dashboards prets a l'emploi, mettre en place le RBAC (role et utilisateur analyste a moindre privilege), faire fonctionner l'alerting, tuner une regle bruyante, creer quatre regles maison, executer une requete de correlation EQL, et appliquer la logique detection-as-code (export, versioning).

Objectifs pedagogiques

- Importer des Detection Rules et des dashboards dans Elastic Cloud.
- Gerer le RBAC : creer un role et un utilisateur analyste.
- Faire fonctionner l'alerting (regles activees -> alertes).
- Tuner une regle trop bruyante (seuil, exception).
- Creer ses propres regles (Custom query, Threshold, EQL).
- Appliquer la logique detection-as-code (exporter / versionner).

Perimetre du rapport

Les sept etapes du TP (RBAC, import des regles, import des dashboards, alerting, tuning, regles maison, EQL/detection-as-code) et les trois defis d'autonomie (faux positif, dashboard maison, matrice MITRE ATT&CK) ont ete menes a terme. Ce rapport documente chaque etape, les obstacles reels rencontres (notamment un bug d'import des objets Lens et un defaut de couverture d'index sur les regles maison), ainsi que les corrections apportees.

Partie II : Espace et RBAC

Choix du Space

Conformement a la recommandation de l'enonce ("le plus simple pour eviter tout blocage : rester dans le Space Default"), aucun Space dedie n'a ete cree : toute la configuration de ce TP (roles, utilisateur, regles, dashboards) reside dans le Space Default, qui reutilise directement la Data View soc-lab-events creee au TP2.

Creation du role soc_analyst_readonly

Un role au moindre privilege a ete cree via Stack Management > Security > Roles. La premiere tentative a introduit une erreur de comprehension de l'interface : les features Kibana (Discover, Dashboard, Security) avaient ete saisies comme un second bloc de privileges Elasticsearch (index privileges) au lieu de la section distincte "Couche applicative" (Kibana privileges). L'erreur a ete corrigee en supprimant ce bloc et en utilisant le bouton "Affecter a l'espace", qui a permis de configurer un controle granulaire par sous-fonctionnalite (le plan trial de ce cluster n'offre pas de bascule globale par groupe : le toggle principal de la categorie Security ne descend pas automatiquement sur ses sous-fonctionnalites comme Alerts, Cases ou Timeline, chacune devant etre reglee individuellement).

Couche	Ressource	Droit accorde
Elasticsearch (index)	soc-lab-events-*, logs-*	read, view_index_metadata
Kibana (feature)	Discover	Read
Kibana (feature)	Dashboard	Read
Kibana (feature)	Security > Alerts	Read
Kibana (feature)	Toutes les autres fonctionnalites	None

Creation de l'utilisateur soc-analyst

Un utilisateur soc-analyst a ete cree (Stack Management > Users) et associe au role soc_analyst_readonly. La creation et la configuration ont ete verifiees a l'ecran a chaque etape avant validation finale.

Reponses aux questions 3

Q3.1 - Pourquoi un utilisateur dedie plutot que elastic ? Le compte elastic est un superutilisateur avec des privileges illimites sur le cluster entier (configuration, suppression d'index, gestion des utilisateurs...). L'utiliser au quotidien pour du triage d'alertes viole le principe de moindre privilege, rend impossible toute tracabilite individuelle des actions (tous les logs porteraient le meme compte) et expose un risque disproportionne si les identifiants fuitent.

Q3.2 - Moindre privilege applique a un analyste L1 ? Un analyste L1 doit pouvoir consulter les evenements (Discover), visualiser les dashboards et lire les alertes de securite -- rien de plus. Il n'a pas besoin de modifier les regles de detection, de gerer les utilisateurs ou d'administrer le cluster. Le role soc_analyst_readonly materialise exactement ce perimetre : lecture seule sur les index de donnees, et acces Kibana limite a Discover / Dashboard / Alerts en lecture.

Point de controle 1

Un utilisateur soc-analyst existe, associe au role soc_analyst_readonly dont les droits (lecture seule sur les index SOC-LAB et sur les fonctionnalites Discover, Dashboard et Alertes) ont ete verifies a l'ecran.

Partie III : Import des regles et des dashboards

Import des 15 regles de detection

Les regles ont ete importees via Securite > Regles > Importer des regles, en choisissant deliberement le fichier soc-lab-detection-rules-cloud.ndjson plutot que la version standard soc-lab-detection-rules.ndjson. Les deux fichiers ont ete inspectes prealablement : leur seule difference structurelle est le champ from de chaque regle (now-6m / now-11m / now-16m dans la version standard, uniformement now-90d dans la version cloud). Cette difference est determinante : les donnees du TP2 ont ete uploadees dans le passe (datees du jour de l'upload), non generees en flux temps reel. Une regle fenetree sur now-6m ne reinterrogerait donc jamais ces evenements et ne declencherait aucune alerte. La version cloud, avec une fenetre de recuperation elargie a 90 jours, couvre le dataset uploade.

Les 15 regles prefixees SOC-LAB ont ete verifiees comme presentes et actives apres import (couvrant Threshold, EQL, Query et une diversite de severites de medium a critical).

Reponses aux questions 4

Q4.1 - Pourquoi deux regles de brute force (Threshold et EQL) ? La regle Threshold (#1, SSH Brute Force) compte un volume d'echecs d'authentification par IP source sur une fenetre donnee : elle detecte l'intensite de l'attaque (combien de tentatives), independamment de son issue. La regle EQL (#2, Brute Force suivi d'un succes) cherche une sequence temporelle precise -- un echec suivi d'un succes pour la meme IP dans les 15 minutes -- et detecte donc specifiquement une compromission reussie, un signal de bien plus haute priorite que le seul volume de tentatives.

Q4.2 - Quelle regle genere le plus de bruit ? D'apres l'analyse empirique des 124 alertes generees (Partie IV), la regle SSH Brute Force (Threshold, seuil initial de 10 sur 5 minutes) etait la plus bruyante avant tuning : son seuil relativement bas et le compte de service deploy generant un volume d'echecs legitimes expliquaient une part disproportionnee des alertes, ce qui justifie precisement le tuning effectue en Partie IV.

Point de controle 2

15 regles prefixees SOC-LAB sont importees et visibles dans Securite > Regles.

Import des 6 dashboards -- obstacle rencontre

L'import des dashboards (Stack Management > Saved Objects > Import, fichier soc-lab-dashboards.ndjson, 48 objets dont 42 visualisations Lens et 6 dashboards) a echoue de maniere constante avec une erreur generique "Internal Server Error". Un diagnostic methodique par isolation a ete mene pour en identifier la cause :

- Import du fichier complet (48 objets) : echec.
- Import des 6 dashboards seuls (sans les 42 objets Lens) : succes.
- Import des 42 objets Lens seuls : echec identique.
- Import d'un seul objet Lens extrait : echec identique.
- Meme objet avec le champ coreMigrationVersion retire : echec identique.
- Objet Lens minimal ecrit a la main : echec identique.

- Export puis reimport d'un objet existant quelconque (test de controle) : succes, confirmant que le mecanisme d'import lui-meme fonctionne.

Cette isolation converge vers une conclusion : le type d'objet sauvegarde lens est specifiquement casse a l'import sur ce cluster, independamment de son contenu -- un probleme vraisemblablement lie a une incompatibilite de version (les objets exportes portent un coreMigrationVersion 8.15.3, tandis que le cluster tourne en version 9.5.5) ou a l'incident "Partial system outage" signale sur la console Elastic Cloud au debut de la session. Strategie retenue : les 6 dashboards (sans panneaux Lens) ont ete importes avec succes, et la visualisation necessaire au Defi B a ete reconstruite manuellement via l'editeur Lens plutot que par import (voir Partie VII).

Reponses aux questions 5

Q5.1 - Que se passe-t-il a l'import si la Data View n'existe pas ? Les dashboards s'importent sans erreur bloquante mais referencent un identifiant de Data View introuvable : leurs panneaux restent vides (aucune donnee ne peut etre resolue). C'est pourquoi la Data View soc-lab-events (ID personnalise, cree au TP2) est un prerequis explicite avant l'import.

Q5.2 - Comment exporter a nouveau un dashboard modifie ? Stack Management > Saved Objects, cocher le ou les dashboards modifies, puis utiliser l'action "Export" -- ce qui produit un fichier NDJSON pouvant etre reimporte ailleurs ou verse dans un depot Git, exactement selon la meme logique que l'export des regles de detection vu en Partie VI.

Point de controle 3

Les 6 dashboards sont importes. Limitation documentee : leurs panneaux Lens sont vides du fait du bug d'import identifie ci-dessus ; un dashboard equivalent fonctionnel a ete reconstruit manuellement (Defi B, Partie VII).

Partie IV : Alerting et tuning

Activation des regles et premiere alerte

Les 15 regles importees ont ete activees. Une regle d'injection SQL (SOC-LAB - Tentative d'injection SQL) affichait un statut "Failed". Le diagnostic via l'onglet Resultats d'execution a revele une erreur de syntaxe KQL precise : Expected "", AND, OR but "(" found, provoquee par des parentheses litterales non echappees dans la requete (url.full:*sleep(*). En KQL, la parenthese est un caractere reserve de groupement logique ; une occurrence litterale dans un motif wildcard doit etre echappee par un backslash. La requete corrigee (url.full:*sleep\(* or url.full:*benchmark\(*) a ete enregistree et la regle est repassee au statut "succeeded".

Apres un cycle d'execution (intervalle de 5 minutes sur la fenetre elargie now-90d), 124 alertes ont ete generees et consultees dans Securite > Alertes.

Reponses aux questions 6

Q6.1 - Quelles regles se sont declenchees, sur quels hotes/IP ? La repartition empirique des 124 alertes a ete extraite par regle et par host.name / source.ip, confirmant que la regle SSH Brute Force (Threshold) etait la plus bruyante, suivie des regles liees au compte de service deploy et a l'IP 198.51.100.88 deja identifiee comme source suspecte au TP2.

Q6.2 - Difference entre une regle et une alerte ? Une regle est une definition statique (requete, planification, seuil, severite) qui s'execute periodiquement. Une alerte est le resultat concret d'une execution de regle qui a trouve une correspondance : c'est un document a part entiere, horodate, associe a des champs d'enrichissement specifiques, qu'un analyste investigue et fait evoluer dans un workflow de triage.

Q6.3 - Quels champs Kibana ajoute-t-il a une alerte ? Un prefixe kibana.alert.* enrichit chaque alerte : kibana.alert.rule.name et kibana.alert.rule.uuid (identite de la regle a l'origine), kibana.alert.severity et kibana.alert.risk_score (criticite), kibana.alert.status et kibana.alert.workflow_status (etat de traitement, ouvert/ferme/acquitte).

Point de controle 4

Au moins une alerte est visible (124 au total) ; les regles desactivees apres observation ont ete verifiees via le filtre "Regles desactivees" (affichage 1-15 sur 15), confirmant que les horodatages de derniere execution visibles etaient des donnees historiques et non de nouvelles executions.

Etape 5 : tuning de la regle SSH Brute Force

La regle SOC-LAB - SSH Brute Force (multiples echecs par IP source) etait identifiee comme trop bruyante, generant des alertes pour des echecs legitimes du compte de service deploy. Deux ajustements ont ete appliques : le seuil a ete eleve de 10 a 20 echecs sur 5 minutes, et une exception a ete ajoutee sur le champ user.name egal a deploy. Une premiere tentative d'edition s'est portee par erreur sur la regle EQL de brute force (SOC-LAB - Brute Force suivi d'une connexion reussie), dont le nom commence par les memes mots -- l'erreur a ete identifiee via la banniere "cette regle contient une instruction de sequence EQL" et corrigee en revenant a la bonne regle Threshold avant de finaliser le tuning.

Reponses aux questions 7

Q7.1 - Reduire les faux positifs ou augmenter la couverture ? Les deux ne sont pas mutuellement exclusifs, mais dans ce cas precis la priorite va a la reduction des faux positifs : une regle qui alerte en continu sur une activite legitime et identifiee (le compte de service deploy) epuise l'attention de l'analyste (fatigue d'alerte) et masque les vrais signaux. Augmenter la couverture n'a de valeur que si les alertes generees restent exploitables ; tuner d'abord le bruit existant est le prealable a toute extension de perimetre.

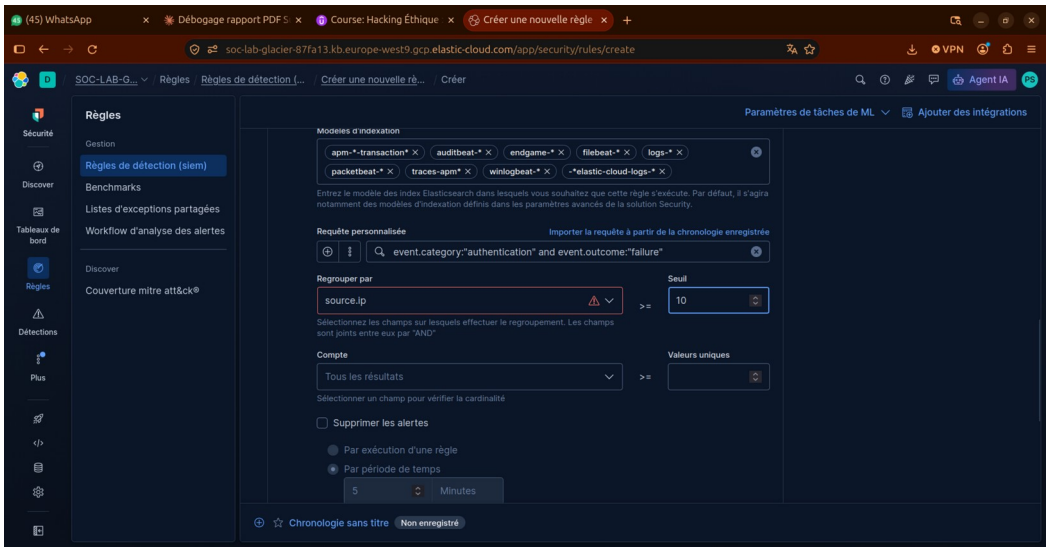
Q7.2 - Une exception est-elle une triche ou une pratique professionnelle ? C'est une pratique professionnelle standard, a condition d'etre documentee et justifiee : l'exception ne desactive pas la detection, elle l'affine en excluant une condition precise et connue (ici, un compte de service dont l'activite a ete tracee et validee comme legitime), tout en continuant a detecter la meme menace pour n'importe quel autre compte. Une exception non justifiee ou trop large serait en revanche un contournement dangereux.

Partie V : Regles maison (Etape 6)

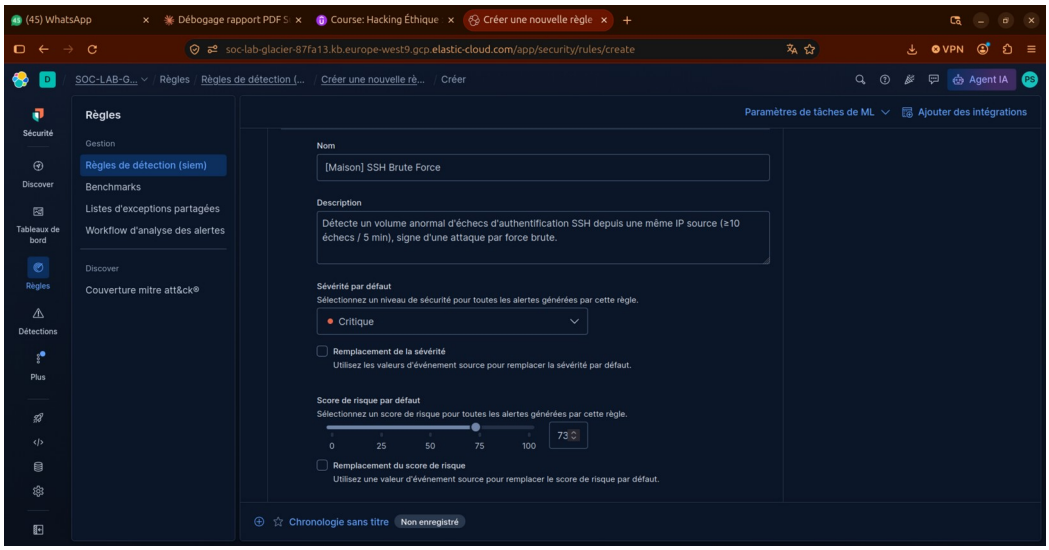
Quatre regles maison ont ete creees via Securite > Regles > Creer une regle > Creation manuelle de regles, toutes avec le prefixe [Maison]. Un obstacle commun a ete rencontre et corrige sur l'ensemble des regles : les modeles d'indexation par defaut du formulaire de creation (apm-*, auditbeat-*, filebeat-*, logs-*, etc.) ne couvrent pas l'index reel des donnees simulees (soc-lab-events-*), cree lors de l'upload des datasets au TP2. Sans cet ajout, les regles s'executent sans erreur mais ne trouvent jamais aucun evenement. Le modele d'indexation soc-lab-events-* a donc ete explicitement ajoute a chacune des quatre regles.

Regle 1 : [Maison] SSH Brute Force (Threshold)

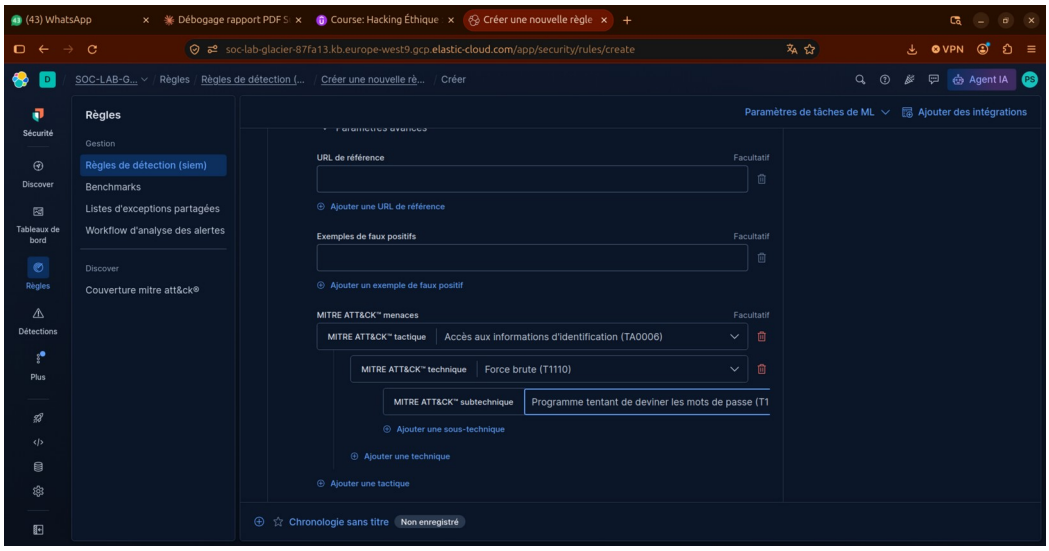
Requete : event.category:"authentication" and event.outcome:"failure", regroupement par source.ip, seuil superieur ou egal a 10 sur une fenetre de 5 minutes. Severite High, score de risque 73. Mapping MITRE ATT&CK : tactique Acces aux informations d'identification (TA0006), technique Force brute (T1110), sous-technique Programme tentant de deviner les mots de passe (T1110.001).



Formulaire de la regle Threshold : requete, regroupement par source.ip, seuil >= 10 sur 5 minutes.



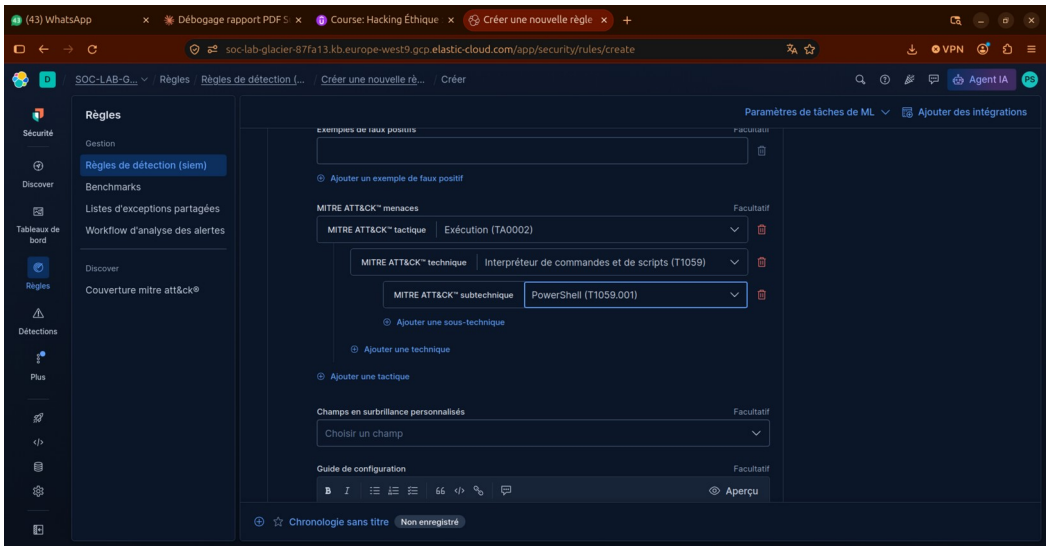
Selection de la tactique MITRE ATT&CK : Acces aux informations d'identification (TA0006).



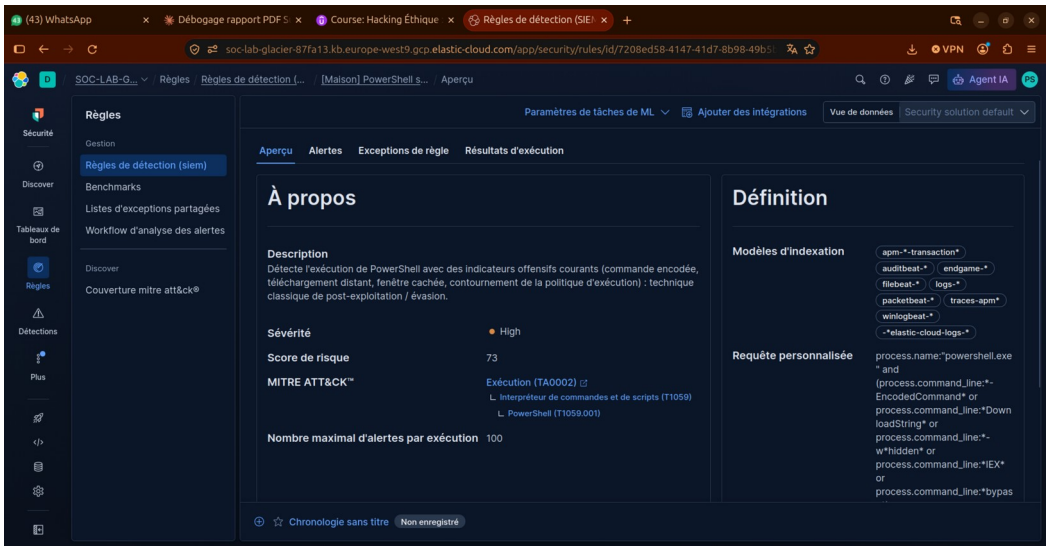
Apercu final de la regle [Maison] SSH Brute Force : severite High, score 73, mapping MITRE complet, planification toutes les 5 minutes.

Regle 2 : [Maison] PowerShell suspect (Custom query)

Requete : process.name:"powershell.exe" and (process.command_line:*-EncodedCommand* or process.command_line:*DownloadString* or process.command_line:*-w*hidden* or process.command_line:*IEX* or process.command_line:*bypass*). Ces motifs sont des indicateurs classiques d'execution PowerShell offensive : commande encodee en base64, telechargement et execution de code distant, fenetre masquee, et contournement de la politique d'execution de scripts. Severite High, score 73. Mapping MITRE ATT&CK : tactique Execution (TA0002), technique Interpreteur de commandes et de scripts (T1059), sous-technique PowerShell (T1059.001).



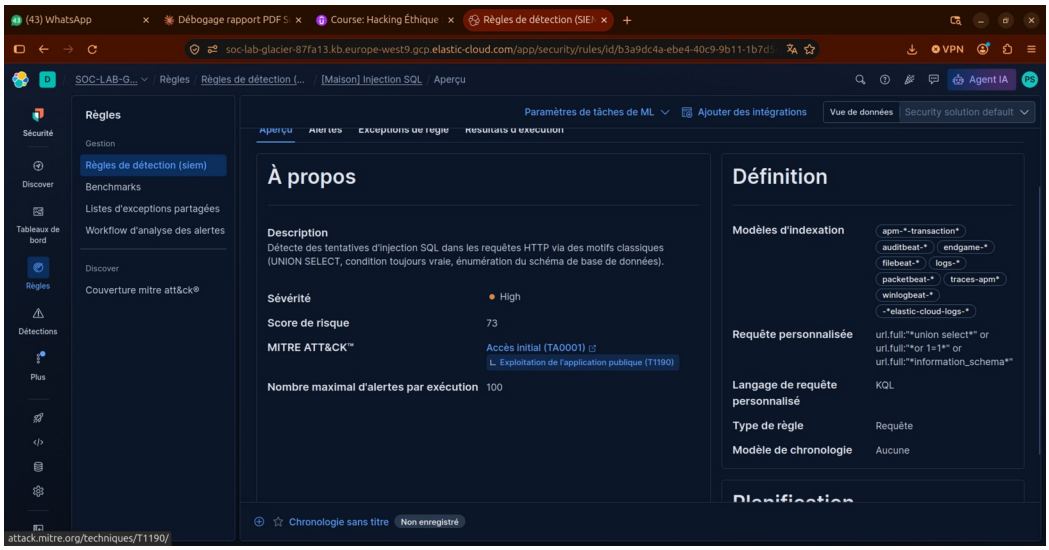
Apercu de la regle [Maison] PowerShell suspect : requete KQL ciblant les indicateurs offensifs PowerShell, severite High, score 73.



Mapping MITRE de la regle PowerShell suspect : Execution (TA0002) > Interpreteur de commandes et de scripts (T1059) > PowerShell (T1059.001).

Regle 3 : [Maison] Injection SQL (Custom query)

Requete : url.full:"*union select*" or url.full:"*or 1=1*" or url.full:"*information_schema*". Une premiere ecriture avec des espaces echappes par backslash (url.full:*or\ 1=1*) a provoque une erreur "Le KQL n'est pas valide", le caractere = cassant le parseur hors guillemets -- la meme classe de probleme que celle rencontree et corrigee a l'Etape 4 sur la regle importee equivalente. La correction retenue encadre chaque motif de guillemets plutot que d'echapper chaque espace individuellement, ce qui traite l'integralite du contenu comme une chaine litterale a comparer en wildcard. Severite High, score 73. Mapping MITRE ATT&CK : tactique Acces initial (TA0001), technique Exploitation de l'application publique (T1190).

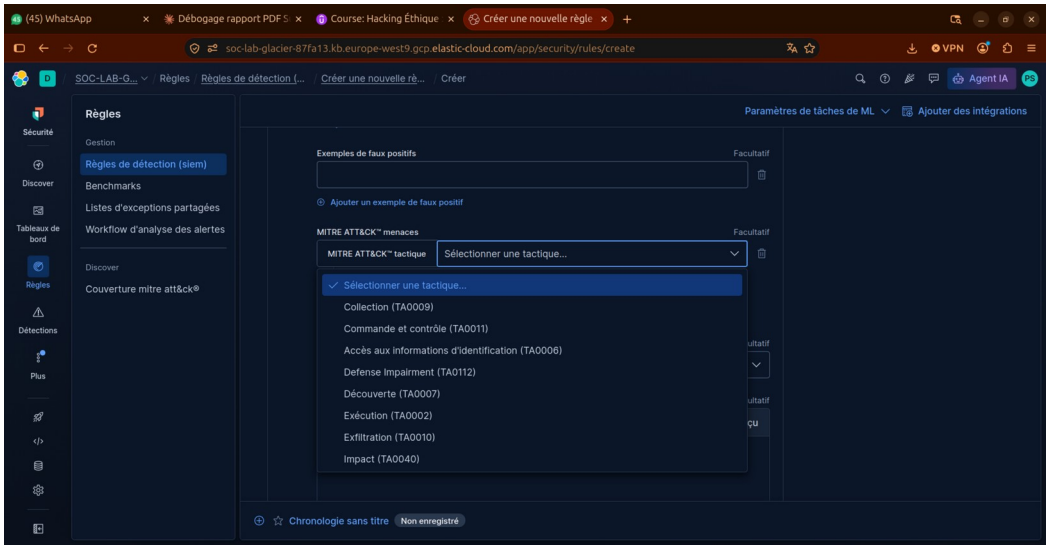


Apercu de la regle [Maison] Injection SQL : requete corrigee entre guillemets, severite High, mapping Acces initial (TA0001) > Exploitation de l'application publique (T1190).

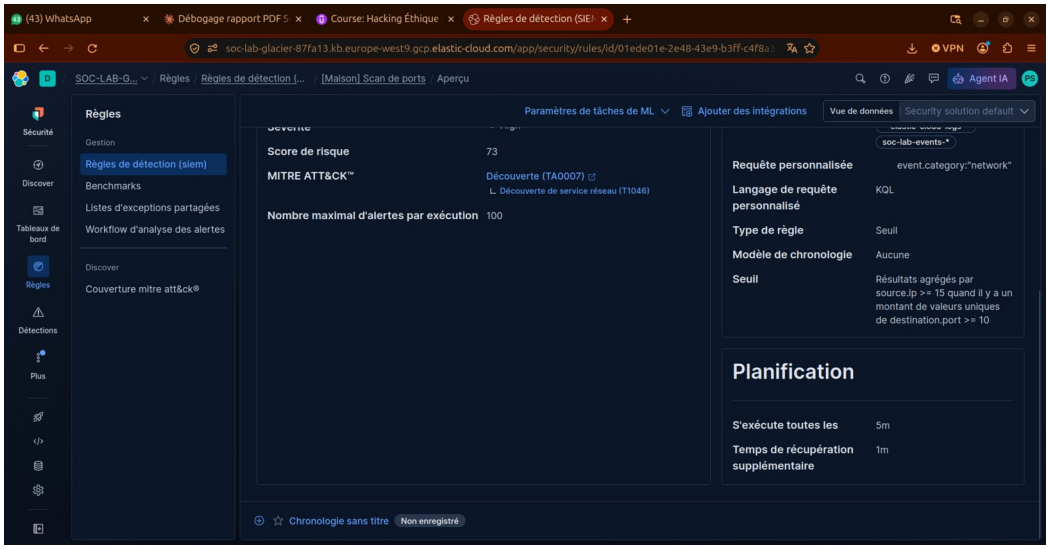
Regle 4 : [Maison] Scan de ports (Threshold + cardinalite)

Requete : event.category:"network", regroupement par source.ip, seuil superieur ou egal a 15 sur 5 minutes, avec un champ de cardinalite sur destination.port (valeurs uniques superieures ou egales a 10). La cardinalite distingue un scan de ports d'un simple pic de trafic : elle ne compte pas le volume d'evenements mais le nombre de ports distincts contactes par une meme IP source, signature caracteristique d'une reconnaissance reseau.

Un second obstacle a ete rencontre sur cette regle specifiquement : le champ destination.port, pourtant bien present dans les donnees (verifie via Discover, ou il apparait dans la liste des champs disponibles de l'index soc-lab-events-2026.10.19), n'apparaissait pas dans le selecteur de champ de cardinalite tant que le modele d'indexation soc-lab-events-* n'avait pas ete ajoute -- confirmation directe que l'obstacle d'indexation decrit plus haut touchait bien la totalite des regles maison, pas seulement celle-ci. Severite High, score 73. Mapping MITRE ATT&CK : tactique Decouverte (TA0007), technique Decouverte de service reseau (T1046).



Selection de la technique MITRE pour une regle maison : menu deroulant des techniques rattachees a une tactique.



Details de la regle [Maison] Scan de ports : seuil source.ip >= 15, cardinalite destination.port >= 10, planification 5 minutes.

Reponses aux questions 8

Q8.1 - Pourquoi un risk score en plus de la severity ? La severity (low/medium/high/critical) est une classification discrete en quatre niveaux, pratique pour filtrer et trier visuellement. Le risk score (0-100) est une mesure continue qui permet un classement plus fin entre alertes de meme severite et peut etre ajuste dynamiquement par des remplacements bases sur les valeurs de l'evenement source (par exemple un score plus eleve si l'hote cible est un serveur critique), ce que la seule severity ne permet pas d'exprimer.

Q8.2 - Quelle metadonnee est indispensable pour un rapport MITRE ATT&CK ? Le mapping tactique + technique (et sous-technique quand elle existe) rattache a

chaque regle est la metadonnee indispensable : c'est elle qui alimente la matrice de couverture (Partie VII, Defi C) et permet de repondre objectivement a la question "quelles techniques de l'adversaire mon SOC est-il capable de detecter ?".

Point de controle 5

Les 4 regles maison sont creees et activees, chacune avec un mapping MITRE ATT&CK complet et une severite/score de risque coherents.

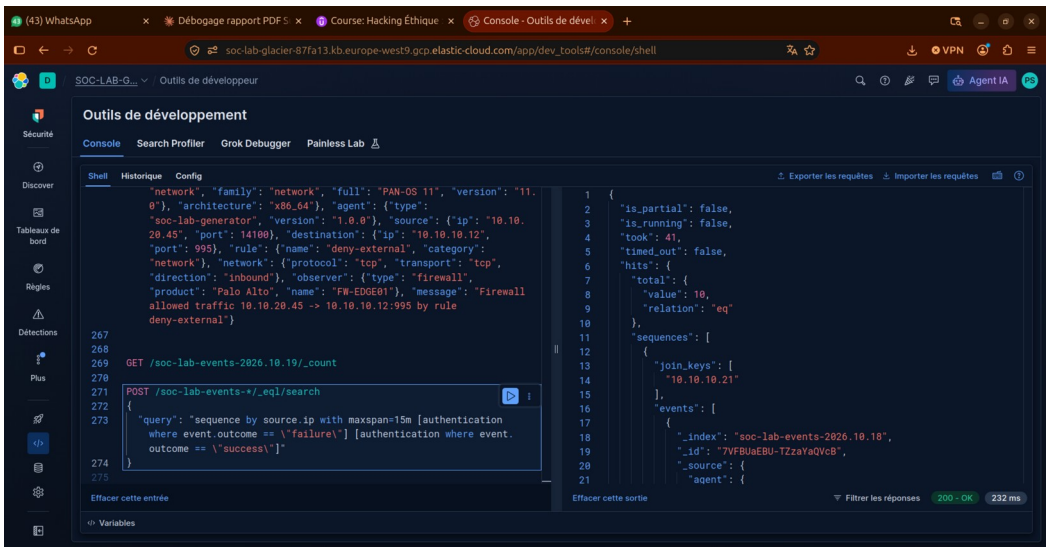
Partie VI : EQL et detection-as-code

Requete de correlation EQL

La requete suivante a ete executee dans Dev Tools :

```
POST /soc-lab-events-*/_eql/search
{
  "query": "sequence by source.ip with maxspan=15m
    [authentication where event.outcome == \"failure\"]
    [authentication where event.outcome == \"success\"]"
}
```

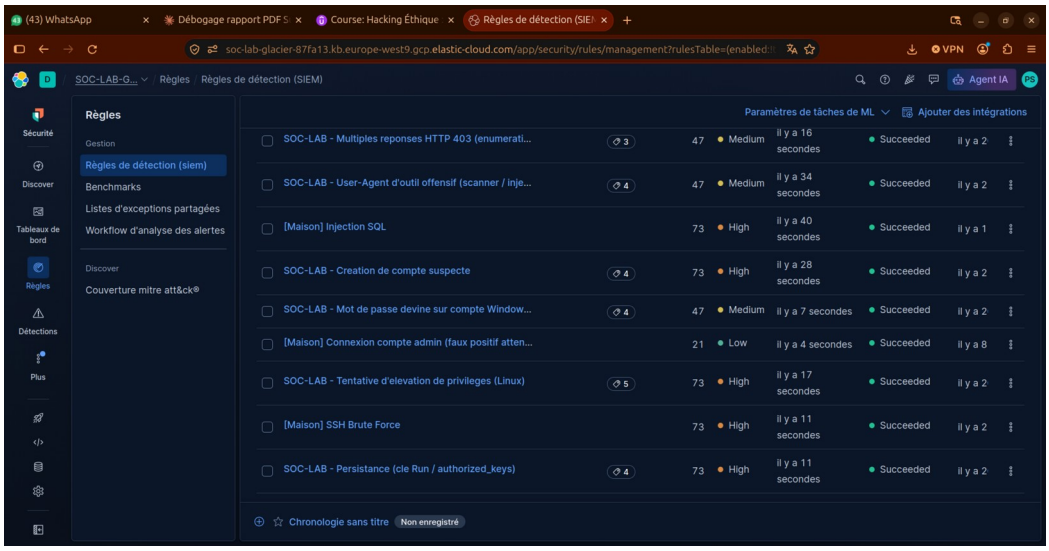
Le resultat a renvoye 10 sequences correspondantes (total.value: 10, relation: eq), chacune identifiant une IP source ayant enchaine un echec d'authentification puis un succes dans la fenetre de 15 minutes -- exactement le scenario couvert par la regle importee EQL (#2, Brute Force suivi d'une connexion reussie). Ces 10 sequences constituent des candidats prioritaires d'investigation : un echec suivi d'un succes peut etre un faux positif benin (l'utilisateur s'est trompe puis a retape son mot de passe) ou un vrai positif critique (un attaquant ayant fini par deviner ou casser le mot de passe).



Dev Tools : requete EQL de sequence (echec puis succes d'authentification, meme source.ip, maxspan 15 minutes) -- 10 sequences trouvees.

Export des regles (detection-as-code)

Les 19 regles (15 importees + 4 maison) ont ete exportees au format NDJSON via Securite > Regles > Actions groupees > Exporter. Comparee au fichier source soc-lab-detection-rules-cloud.ndjson, cet export differe sur plusieurs points qui materialisent concretement le travail effectue durant le TP : les 4 regles maison portent un rule_id genere par Kibana (UUID) absent du fichier original, et les regles modifiees en cours de TP (Injection SQL avec sa requete corrigee, SSH Brute Force avec son seuil releve a 20 et son exception) presentent un revision superieur a 0 ainsi qu'un contenu divergent -- la preuve tangible du tuning effectue.



Liste des 19 regles installees, toutes au statut Succeeded apres correction des modeles d'indexation.

Reponses aux questions 9

Q9.1 - Qu'apporte l'EQL face au KQL pour la correlation ? KQL recherche des evenements independamment les uns des autres : il ne sait pas relier deux evenements entre eux dans le temps. EQL permet de definir une sequence (sequence by ... with maxspan=...) : plusieurs conditions devant survenir dans un ordre precis, dans une fenetre de temps donnee, pour une meme entite. C'est indispensable pour detecter des scenarios d'attaque multi-etapes (ici, un echec suivi d'un succes) qu'une simple requete KQL ne peut pas exprimer.

Q9.2 - Comment revue de code et CI s'appliquent-elles aux regles de detection ? Une regle de detection est du code structure (JSON/NDJSON) : elle peut donc suivre les memes pratiques qu'un logiciel -- versionnement Git, modification via pull request relue par un pair (pour eviter qu'une regle mal reglee ne genere trop de faux positifs ou, a l'inverse, ne rate des vrais positifs), et tests automatises en integration continue (validation de la syntaxe KQL/EQL, execution contre un jeu de donnees de test pour verifier la detection attendue avant tout deploiement en production). C'est l'essence du detection-as-code.

Versionner les regles dans Git apporte en particulier : tracabilite (qui a modifie quoi, quand, pourquoi), possibilite de rollback si une regle modifiee degrade la detection, revue par les pairs avant deploiement, tests automatisables en CI/CD, et deploiement reproductible et identique sur plusieurs environnements (dev/test/prod).

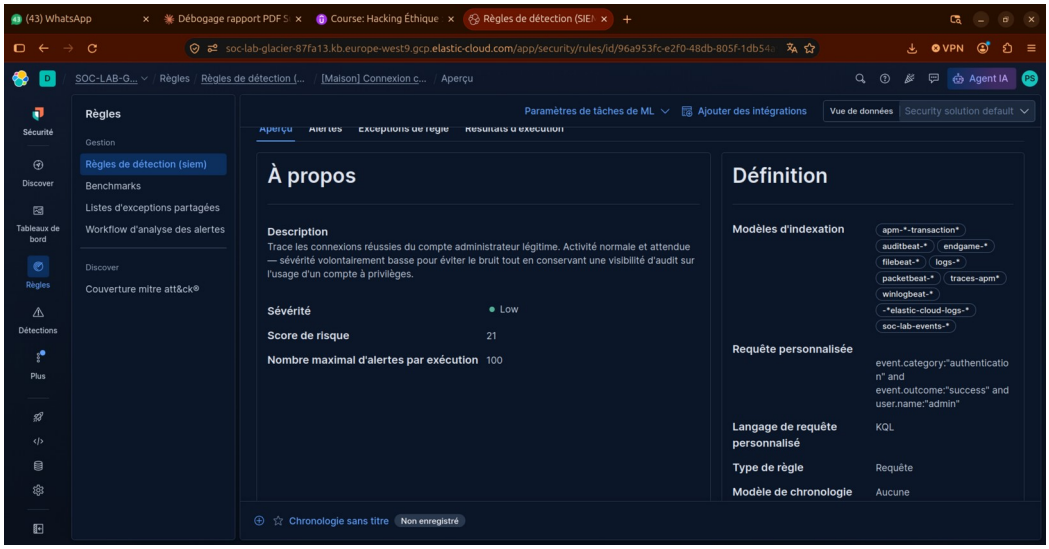
Point de controle

Une requete EQL de correlation a ete executee avec succes (10 sequences trouvees) et les 19 regles ont ete exportees en NDJSON, demontrant la comprehension pratique de la logique detection-as-code.

Partie VII : Defis (autonomie)

Defi A : faux positif documente

La regle [Maison] Connexion compte admin (faux positif attendu) a ete creee avec la requete event.category:"authentication" and event.outcome:"success" and user.name:"admin", tracant toute connexion reussie du compte administrateur. Contrairement aux quatre regles maison precedentes, celle-ci a volontairement ete configuree en severite Low (score de risque 21) : il ne s'agit pas d'une menace mais d'une activite legitime et attendue, qu'un SOC souhaite neanmoins tracer a des fins d'audit sans generer de bruit pour les analystes. Un exemple de faux positif a ete documente dans le champ dedie de la regle ("connexion normale de l'administrateur systeme dans le cadre de ses taches habituelles").



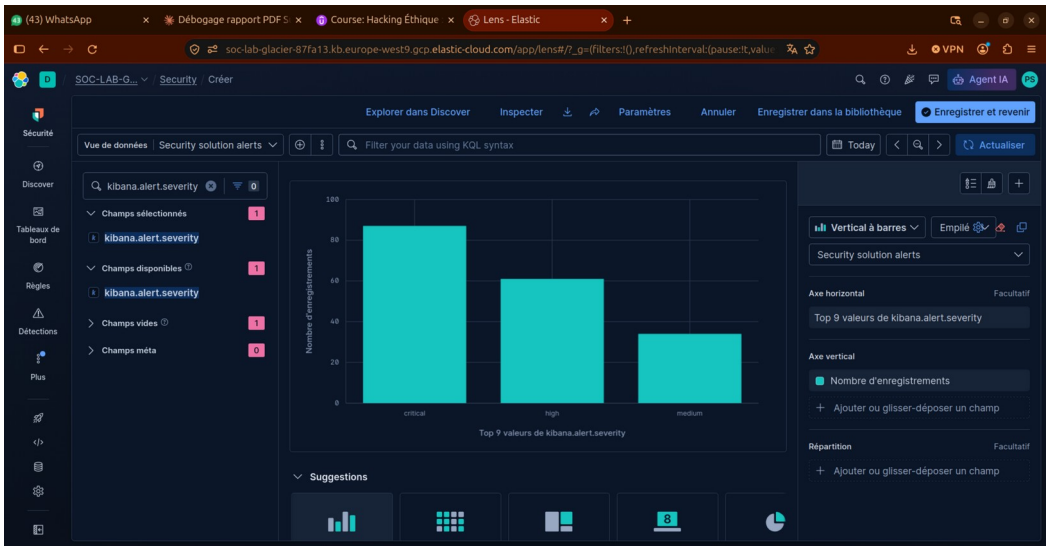
Regle [Maison] Connexion compte admin : severite volontairement Low (score 21), requete ciblant les connexions reussies du compte admin.

Defi B : dashboard maison

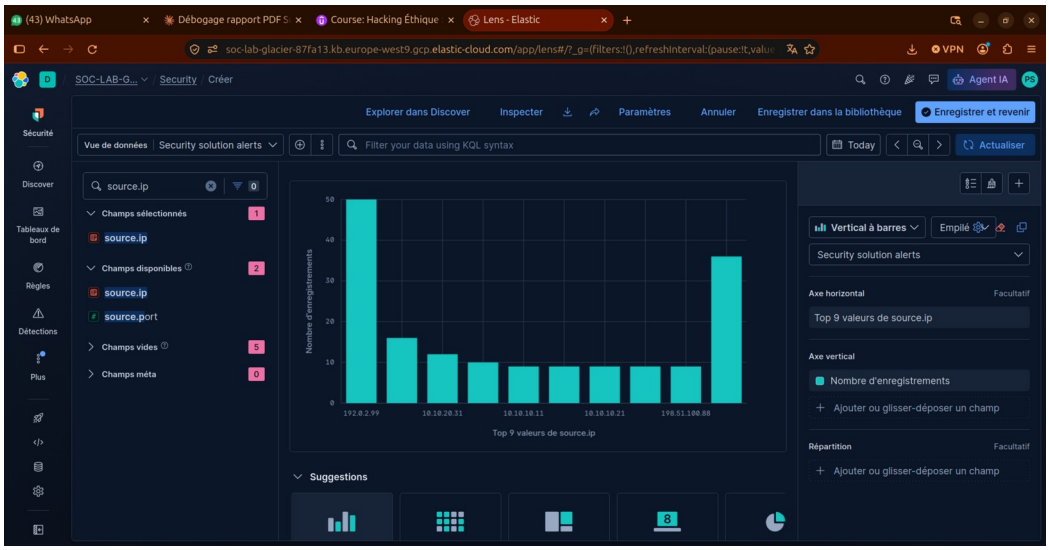
Face au bug d'import des objets Lens documente en Partie III, un dashboard equivalent a ete construit manuellement : [Maison] SOC Overview - Alertes, compose de deux visualisations Lens, toutes deux batties sur la Data View Security solution alerts.

Alertes par severite -- repartition par kibana.alert.severity (nombre d'enregistrements), produisant automatiquement un graphique en barres : critical (~87), high (~61), medium (~34).

Top source.ip -- repartition des alertes par source.ip (top 9), faisant ressortir nettement une adresse (192.0.2.99, ~50 alertes) loin devant les suivantes (198.51.100.88, ~36), candidate prioritaire a l'investigation.



Visualisation Lens Alertes par severite : critical ~87, high ~61, medium ~34.



Visualisation Lens Top source.ip : 192.0.2.99 nettement en tete (~50 alertes), suivie de 198.51.100.88 (~36).



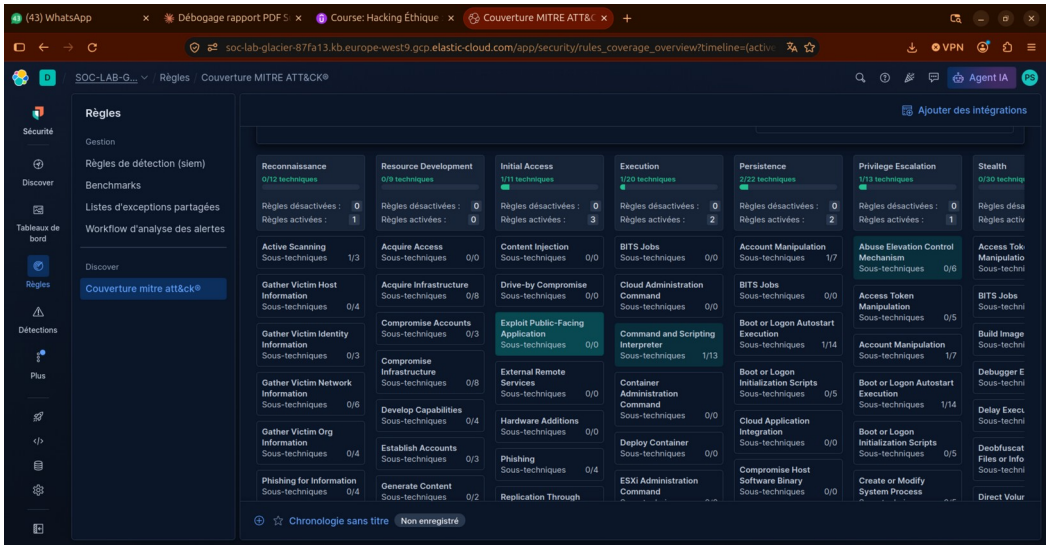
Dashboard final [Maison] SOC Overview - Alertes, les deux visualisations reunies et sauvegardees.

Defi C : matrice de couverture MITRE ATT&CK

Le mapping MITRE ATT&CK de chacune des 4 regles maison a ete recapitule puis verifie visuellement via la page native Securite > Regles > Couverture MITRE ATT&CK, qui confirme la couverture declaree.

Regle maison	Tactique	Technique
SSH Brute Force	Acces aux informations d'identification (TA0006)	Force brute (T1110) > Password Guessing (T1110.001)
PowerShell suspect	Execution (TA0002)	Interpreteur de commandes et de scripts (T1059) > PowerShell (T1059.001)
Injection SQL	Acces initial (TA0001)	Exploitation de l'application publique (T1190)
Scan de ports	Decouverte (TA0007)	Decouverte de service reseau (T1046)

Cette repartition sur quatre tactiques distinctes (Acces initial, Execution, Acces aux identifiants, Decouverte) montre une couverture de detection etalee sur plusieurs phases d'une kill chain plutot que concentree sur un seul type de menace -- une analyse de couverture typique d'un SOC.



Matrice de couverture MITRE ATT&CK native de Kibana : les techniques couvertes par les regles maison et importees apparaissent surlignees (Exploit Public-Facing Application, Command and Scripting Interpreter, etc.).

Point de controle defis

Les trois defis ont ete menes a terme : une regle de faux positif documentee et volontairement classee en severite basse (Defi A), un dashboard maison fonctionnel compense l'echec d'import des visualisations Lens (Defi B), et la matrice de couverture MITRE ATT&CK confirme visuellement le mapping des 4 regles maison sur 4 tactiques distinctes (Defi C).

Conclusion

Ce TP4 a transforme la plateforme SOC-LAB-GLACIER, jusque-la capable de recherche manuelle (TP2), en un SOC produisant des alertes exploitables : RBAC au moindre privilege, import de 15 regles et de 6 dashboards, alerting fonctionnel (124 alertes observees), tuning argumente d'une regle bruyante, creation de 4 regles maison couvrant 4 tactiques MITRE ATT&CK distinctes, correlation EQL demontree et export detection-as-code effectue.

Plusieurs obstacles reels ont ete rencontres et documentes plutot que masques : une confusion d'interface entre privileges Elasticsearch et privileges Kibana lors de la creation du role, un bug d'import constant et methodiquement isole sur les objets Saved Object de type lens (contourne par une reconstruction manuelle du dashboard du Defi B), une erreur de syntaxe KQL sur des parentheses non echappees corrigee sur une regle importee puis anticipee sur une regle maison equivalente, un oubli initial du modele d'indexation soc-lab-events-* sur les premieres regles maison (silencieusement sans alerte generee, corrige sur les quatre regles), et une erreur d'aiguillage entre deux regles de brute force au nom similaire lors du tuning.

La plateforme dispose desormais de 19 regles de detection actives, d'un dashboard d'alertes fonctionnel, d'une matrice de couverture MITRE ATT&CK verifiee et d'un export NDJSON pret a etre verse dans un depot Git pour amorcer une pratique de detection-as-code. C'est sur cette base operationnelle que s'appuiera le TP5, consacre au triage, a l'investigation, a la correlation et a la redaction d'un rapport d'incident a partir des alertes desormais generees.